

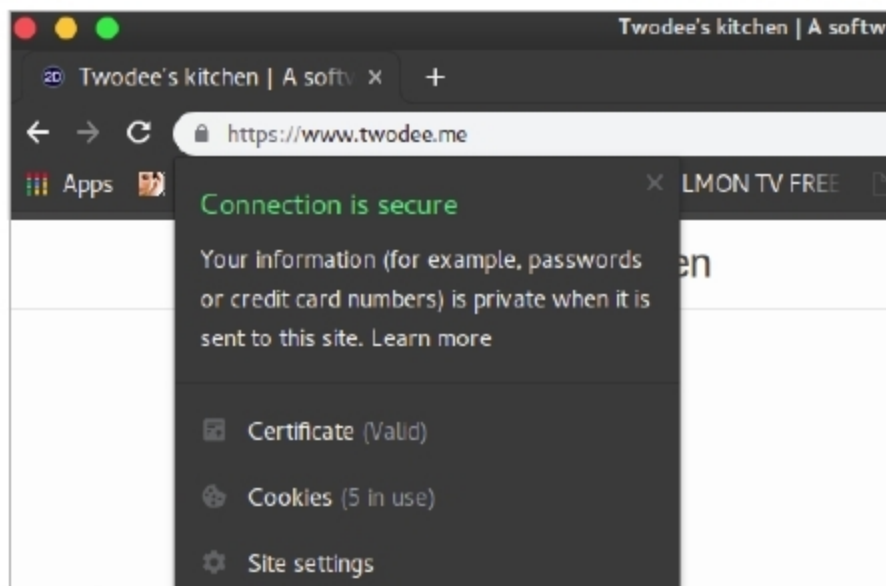


Figure 2: HTTPS enabled website

or Digicert. You can also get domain-validated certificates from Letsencrypt for free.

Apache uses the *mod_ssl* module to support SSL certificates. Use the *default_ssl* configuration file for ease:

```
sudo ln -s /etc/apache2/sites-available/default-ssl.conf /
etc/apache2/sites-enabled/your-site-ssl.conf
```

Once you get the signed SSL certificate, you can add it to your new virtualhost configuration:

```
<VirtualHost *:443>
...
SSLEngine on
SSLCertificateFile /path/to/your/crt/file.crt
SSLCertificateKeyFile /path/to/your/key/file.key
SSLCertificateChainFile /path/to/your/bundle/file.crt
...
</VirtualHost>
```

Finally, enable the SSL module:

```
sudo a2enmod ssl
sudo systemctl restart apache2
```

Securing MySQL

Although setting up the security of MySQL can be done at a later time, after installation, we should ideally start installing it in the most secure way possible.

We start off by installing MySQL by using the following command:

```
sudo apt-get install mysql-server
```

Then we set up the *mysql-server* by creating a directory layout for our databases:

```
sudo mysql_install_db
```

After that, we run the following command to run a wizard that will help us remove some potentially dangerous defaults. This will remove the ability for anyone to log into MySQL by default, disable logging in remotely with the administrator account, and remove some test databases that are insecure.

```
sudo mysql_secure_installation
```

The MySQL configuration file: The main configuration file for MySQL is located at */etc/mysql/my.cnf* in Ubuntu. Tweaking this, we can lock down the MySQL server. In the *[mysqld]* section, set the bind address to the local loopback network device. This will make sure only this machine can access this MySQL instance and not remotely.

```
bind-address = 127.0.0.1
```

We will now shut off a function that allows access to the underlying file system from within MySQL, which can be potentially dangerous if allowed, by using the following command:

```
local-infile=0
```

This will disable loading files from the file system from within MySQL.

Logging: Logging is important to check for events and suspicious activities. This is usually enabled by default, but we can set where the log file is stored, as follows:

```
log=/var/log/mysql-log
```

We should also make sure that neither other users without high privileges, nor the public, are able to view the file.

User access control: As per our previous rule of thumb, we should provide users access to only those resources that are absolutely necessary.

For instance, suppose we have a user called 'noob' that we want to allow read access to, because noob – being a noob, makes a lot of mistakes and we cannot allow that in production. Noob is given an isolated database - MirrorDB - which mirrors the actual production database to teach the structure to it.

Creating the noob user: Use the following command to create the noob user:

```
CREATE USER 'noob'@'localhost' IDENTIFIED BY 'noobpassword';
```

And then we grant the necessary permissions, as follows:

```
GRANT SELECT ON MirrorDB.* TO 'noob'@'localhost';
```

If we want to revoke noob's access, we use the following command:

```
REVOKE SELECT ON MirrorDB.* FROM 'noob'@'localhost';
```